

Документация по архитектуре набора инструкций RISC-IV (ISA)

ISA RISC-IV — это простая регистровая архитектура набора инструкций, вдохновлённая RISC-V. В этой документации приводится обзор инструкций, доступных в RISC-IV ISA, их синтаксис и семантика.

Обзор архитектуры

RISC-IV — это 32-битная RISC-архитектура (Reduced Instruction Set Computer), вдохновлённая спецификацией RISC-V. Она включает:

- 32 регистра общего назначения (включая один жёстко зафиксированный нулевой регистр);
- инструкции фиксированной длины 4 байта;
- load-store архитектуру (доступ к памяти только через специальные инструкции);
- простые режимы адресации;
- ввод-вывод с отображением в память (Memory-Mapped I/O);
- поддержку вызовов и возвратов функций через инструкции jump-and-link;
- арифметические, логические операции и операции управления потоком.

Эта архитектура предоставляет чистую и ортогональную систему команд, хорошо демонстрирующую принципы RISC-дизайна, и отлично подходит для учебных целей, оставаясь при этом достаточно мощной для практических задач.

Комментарии в ассемблерном коде RISC-IV обозначаются символом `;`.

Вдохновлено [RISC-V](#)

Директивы релокации непосредственных значений

В языке ассемблера RISC-IV есть специальные директивы для работы с большими непосредственными значениями, которые не помещаются в стандартные форматы инструкций:

- **%hi(symbol)**
 - **Описание:** Используется для извлечения старших 20 бит 32-битного адреса или непосредственного значения.
 - **Использование:** `lui rd, %hi(symbol)`
 - **Операция:** `rd <- (symbol & 0xFFFFF000)`
- **%lo(symbol)**
 - **Описание:** Используется для извлечения младших 12 бит 32-битного адреса или непосредственного значения.
 - **Использование:** `addi rd, rs, %lo(symbol)`
 - **Операция:** `rd <- rs + (symbol & 0x00000FFF)`

Обычно эти директивы используются вместе, чтобы загрузить полный 32-битный адрес в регистр:

```
lui a0, %hi(address) ; Загрузить старшие 20 бит в a0
addi a0, a0, %lo(address) ; Добавить младшие 12 бит к a0
```

Инструкции

Размер инструкции: 4 байта.

Инструкции перемещения данных

- **Load Upper Immediate**
 - **Синтаксис:** `lui <rd>, <k>`
 - **Описание:** Загрузить непосредственное значение, сдвинутое влево на 12 бит, в регистр назначения.
 - **Операция:** `rd <- k << 12`
- **Move**
 - **Синтаксис:** `mv <rd>, <rs>`
 - **Описание:** Скопировать значение из регистра-источника в регистр назначения.
 - **Операция:** `rd <- rs`
- **Store Word**
 - **Синтаксис:** `sw <rs2>, <offset>(<rs1>)`
 - **Описание:** Сохранить значение из регистра-источника в память по адресу, вычисленному как сумма смещения и базового регистра.
 - **Операция:** `M[offset + rs1] <- rs2`
- **Store Byte**
 - **Синтаксис:** `sb <rs2>, <offset>(<rs1>)`
 - **Описание:** Сохранить младшие 8 бит значения из регистра-источника в память по адресу, вычисленному как сумма смещения и базового регистра.
 - **Операция:** `M[offset + rs1] <- rs2 & 0xFF`
- **Load Word**
 - **Синтаксис:** `lw <rd>, <offset>(<rs1>)`
 - **Описание:** Загрузить слово из памяти по адресу, вычисленному как сумма смещения и базового регистра, в регистр назначения.
 - **Операция:** `rd <- M[offset + rs1]`

Арифметические инструкции

- **Add Immediate**
 - **Синтаксис:** `addi <rd>, <rs1>, <k>`
 - **Описание:** Прибавить непосредственное значение к исходному регистру и сохранить результат в регистре назначения. Сохраняются только младшие 12 бит результата (старшие

20 бит зависят от знака непосредственного значения).

- **Операция:** `rd <- rs1 + k`

- **Add**

- **Синтаксис:** `add <rd>, <rs1>, <rs2>`
- **Описание:** Сложить значения двух исходных регистров и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 + rs2`

- **Subtract**

- **Синтаксис:** `sub <rd>, <rs1>, <rs2>`
- **Описание:** Вычесть значение второго исходного регистра из первого и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 - rs2`

- **Multiply**

- **Синтаксис:** `mul <rd>, <rs1>, <rs2>`
- **Описание:** Умножить значения двух исходных регистров и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 * rs2`

- **Multiply High**

- **Синтаксис:** `mulh <rd>, <rs1>, <rs2>`
- **Описание:** Умножить значения двух исходных регистров и сохранить старшую часть результата в регистре назначения.
- **Операция:** `rd <- (rs1 * rs2) >> (word size)`

- **Divide**

- **Синтаксис:** `div <rd>, <rs1>, <rs2>`
- **Описание:** Разделить значение первого исходного регистра на значение второго и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 / rs2`

- **Remainder**

- **Синтаксис:** `rem <rd>, <rs1>, <rs2>`
- **Описание:** Вычислить остаток от деления первого исходного регистра на второй и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 % rs2`

Побитовые инструкции

- **Logical Shift Left**

- **Синтаксис:** `sll <rd>, <rs1>, <rs2>`
- **Описание:** Сдвинуть значение первого исходного регистра влево на число бит, заданное вторым исходным регистром, и сохранить результат в регистре назначения.

- **Операция:** `rd <- rs1 << rs2`

- **Logical Shift Right**

- **Синтаксис:** `srl <rd>, <rs1>, <rs2>`
- **Описание:** Сдвинуть значение первого исходного регистра вправо на число бит, заданное вторым исходным регистром, и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 >> rs2`

- **Arithmetic Shift Right**

- **Синтаксис:** `sra <rd>, <rs1>, <rs2>`
- **Описание:** Сдвинуть значение первого исходного регистра вправо на число бит, заданное вторым исходным регистром, с сохранением знака и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 >> rs2`

- **Bitwise AND**

- **Синтаксис:** `and <rd>, <rs1>, <rs2>`
- **Описание:** Выполнить побитовое И над значениями двух исходных регистров и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 & rs2`

- **Bitwise OR**

- **Синтаксис:** `or <rd>, <rs1>, <rs2>`
- **Описание:** Выполнить побитовое ИЛИ над значениями двух исходных регистров и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 | rs2`

- **Bitwise XOR**

- **Синтаксис:** `xor <rd>, <rs1>, <rs2>`
- **Описание:** Выполнить побитовое исключающее ИЛИ над значениями двух исходных регистров и сохранить результат в регистре назначения.
- **Операция:** `rd <- rs1 ^ rs2`

Инструкции управления потоком

- **Jump**

- **Синтаксис:** `j <k>`
- **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения.
- **Операция:** `pc <- pc + k`

- **Jump and Link**

- **Синтаксис:** `jal <rd>, <k>`
- **Описание:** Сохранить адрес следующей инструкции в регистре назначения и перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения.

- **Операция:** `rd <- pc + 4, pc <- pc + k`
- **Jump Register**
 - **Синтаксис:** `jr <rs>`
 - **Описание:** Перейти по адресу, хранящемуся в исходном регистре.
 - **Операция:** `pc <- rs`
- **Branch if Equal to Zero**
 - **Синтаксис:** `beqz <rs1>, <k>`
 - **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если значение исходного регистра равно нулю.
 - **Операция:** `if rs1 == 0 then pc <- pc + k`
- **Branch if Not Equal to Zero**
 - **Синтаксис:** `bnez <rs1>, <k>`
 - **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если значение исходного регистра не равно нулю.
 - **Операция:** `if rs1 != 0 then pc <- pc + k`
- **Branch if Greater Than**
 - **Синтаксис:** `bgt <rs1>, <rs2>, <k>`
 - **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если значение первого исходного регистра больше значения второго.
 - **Операция:** `if rs1 > rs2 then pc <- pc + k`
- **Branch if Less Than or Equal**
 - **Синтаксис:** `ble <rs1>, <rs2>, <k>`
 - **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если значение первого исходного регистра меньше или равно значению второго.
 - **Операция:** `if rs1 <= rs2 then pc <- pc + k`
- **Branch if Greater Than (Unsigned)**
 - **Синтаксис:** `bgtu <rs1>, <rs2>, <k>`
 - **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если беззнаковое значение первого исходного регистра больше беззнакового значения второго.
 - **Операция:** `if rs1 > rs2 then pc <- pc + k`
- **Branch if Less Than or Equal (Unsigned)**
 - **Синтаксис:** `bleu <rs1>, <rs2>, <k>`
 - **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если беззнаковое значение первого исходного регистра

меньше или равно беззнаковому значению второго.

- **Операция:** `if rs1 <= rs2 then pc <- pc + k`

- **Branch if Equal**

- **Синтаксис:** `beq <rs1>, <rs2>, <k>`
- **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если значения двух исходных регистров равны.
- **Операция:** `if rs1 == rs2 then pc <- pc + k`

- **Branch if Not Equal**

- **Синтаксис:** `bne <rs1>, <rs2>, <k>`
- **Описание:** Перейти по адресу, вычисленному как сумма текущего счётчика команд и непосредственного значения, если значения двух исходных регистров не равны.
- **Операция:** `if rs1 != rs2 then pc <- pc + k`

- **Halt**

- **Синтаксис:** `halt`
- **Описание:** Остановить машину.

Специфичные представления состояния ISA

- `<reg>:dec`, `<reg>:hex` — просмотр значения конкретного регистра в десятичном или шестнадцатеричном формате.

Доступные регистры: `Zero`, `Ra`, `Sp`, `Gp`, `Tr`, `T0`, `T1`, `T2`, `S0Fp`, `S1`, `A0`, `A1`, `A2`, `A3`, `A4`, `A5`, `A6`, `A7`, `S2`, `S3`, `S4`, `S5`, `S6`, `S7`, `S8`, `S9`, `S10`, `S11`, `T3`, `T4`, `T5`, `T6`.